Recorrido Mínimo Uno a Todos

Técnicas de Diseño de Algoritmos

FCEyN UBA

21 de octubre 2026

Plan de hoy

Recorrido Mínimo

- BFS
- Dijkstra
- Bellman-Ford
- Floyd-Warshall
- DAGs

Ejercicios

- Policías
- Julián y los Mares
- Lautaro y los Monstruos
- Depósitos

Recorrido mínimo

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

BFS

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con BFS?

BFS

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con BFS? **Así nomás, no.**
- ¿Qué tengo que cambiar?

BFS

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con BFS? **Así nomás, no.**
- ¿Qué tengo que cambiar? **El costo tiene que ser el mismo para todas las aristas.**[1]
- ¿Complejidad?

[1] Podemos relajar esto un poco, ver acá.

BFS

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con BFS? **Así nomás, no.**
- ¿Qué tengo que cambiar? **El costo tiene que ser el mismo para todas las aristas.**[1]
- ¿Complejidad? $\boldsymbol{O(|V| + |E|)}$ **sobre lista de adyacencias.**

[1] Podemos relajar esto un poco, ver acá.

Dijkstra

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con Dijkstra?

Dijkstra

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con Dijkstra? **Así nomás, no.**
- ¿Qué tengo que cambiar?

Dijkstra

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con Dijkstra? **Así nomás, no.**
- ¿Qué tengo que cambiar? **El costo tiene que ser ≥ 0.**
- ¿Complejidad?

Dijkstra

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con Dijkstra? **Así nomás, no.**
- ¿Qué tengo que cambiar? **El costo tiene que ser ≥ 0.**
- ¿Complejidad? $\boldsymbol{O(\min\{|E| \log |V|, |V|^2\})}$ **sobre lista de adyacencias.**

Bellman-Ford

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con Bellman-Ford?

Bellman-Ford

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con Bellman-Ford? **Sí!**
- ¿Qué pasa si hay un ciclo con suma de costos negativa?

Bellman-Ford

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con Bellman-Ford? **Sí!**
- ¿Qué pasa si hay un ciclo con suma de costos negativa? **Te lo dice, es crack.**
- ¿Complejidad?

Bellman-Ford

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con Bellman-Ford? **Sí!**
- ¿Qué pasa si hay un ciclo con suma de costos negativa? **Te lo dice, es crack.**
- ¿Complejidad? $O(|V||E|)$ **sobre lista de adyacencias.**

Floyd-Warshall

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con Floyd-Warshall?

Floyd-Warshall

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con Floyd-Warshall? **Sí, pero resuelve algo más general: las distancias entre todos los pares.**
- ¿Qué pasa si hay un ciclo con suma de costos negativa?

Floyd-Warshall

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con Floyd-Warshall? **Sí, pero resuelve algo más general: las distancias entre todos los pares.**
- ¿Qué pasa si hay un ciclo con suma de costos negativa? **También lo detecta: algún $d[u][u]$ queda negativo.**
- ¿Complejidad?

Floyd-Warshall

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con Floyd-Warshall? **Sí, pero resuelve algo más general: las distancias entre todos los pares.**
- ¿Qué pasa si hay un ciclo con suma de costos negativa? **También lo detecta: algún $d[u][u]$ queda negativo.**
- ¿Complejidad? **$O(|V|^3)$ en tiempo y $O(|V|^2)$ en memoria.**
- ¿Cuándo conviene?

Floyd-Warshall

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Se puede resolver con Floyd-Warshall? **Sí, pero resuelve algo más general: las distancias entre todos los pares.**
- ¿Qué pasa si hay un ciclo con suma de costos negativa? **También lo detecta: algún $d[u][u]$ queda negativo.**
- ¿Complejidad? **$O(|V|^3)$ en tiempo y $O(|V|^2)$ en memoria.**
- ¿Cuándo conviene? **Si necesitamos todos los pares: correr Bellman-Ford desde cada vértice costaría $O(|V|^2|E|)$.**

DAGs

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Y si G es un DAG?

DAGs

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Y si G es un DAG? **Recorremos los vértices en orden topológico y relajamos las aristas salientes de cada uno.**
- ¿Y los costos negativos?

DAGs

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Y si G es un DAG? **Recorremos los vértices en orden topológico y relajamos las aristas salientes de cada uno.**
- ¿Y los costos negativos? **No son problema: sin ciclos, no hay ciclos negativos.**
- ¿Complejidad?

DAGs

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Y si G es un DAG? **Recorremos los vértices en orden topológico y relajamos las aristas salientes de cada uno.**
- ¿Y los costos negativos? **No son problema: sin ciclos, no hay ciclos negativos.**
- ¿Complejidad? $\boldsymbol{O(|V| + |E|)}$ **sobre lista de adyacencias.**
- ¿Qué ganamos además?

DAGs

Problema

Sea $G = (V, E)$ un (di)grafo con una función de costo para las aristas $c\colon E \to \mathbb{R}$, y v un vértice de G. Para todo $w \in V$, encontrar la mínima suma del costo de las aristas de un recorrido desde v a w.

- ¿Y si G es un DAG? **Recorremos los vértices en orden topológico y relajamos las aristas salientes de cada uno.**
- ¿Y los costos negativos? **No son problema: sin ciclos, no hay ciclos negativos.**
- ¿Complejidad? $O(|V| + |E|)$ **sobre lista de adyacencias.**
- ¿Qué ganamos además? **Camino máximo también es fácil: multiplicamos los pesos por** -1**.**

Policías

Problema

La nueva reglamentación de una ciudad establece que toda esquina debe estar a lo sumo a 5 cuadras de una estación de policía. Dada la lista de esquinas $\{v_1, \ldots, v_n\}$ de la ciudad, la lista $\{p_1, \ldots, p_k\}$ de esquinas donde hay policías[2], y la lista E de calles, debemos indicar si la normativa se cumple, y en caso contrario cuáles esquinas son las que quedan "desprotegidas".

[2] O sea, asumimos que la policía siempre se ubica en una esquina.

Policías - Resolución

- Primero el modelado: en este caso es bastante directo, las esquinas son los nodos, y las calles los ejes.

Policías - Resolución

- Primero el modelado: en este caso es bastante directo, las esquinas son los nodos, y las calles los ejes.
- Entonces lo que nos pide el enunciado es que todo nodo esté a una distancia menor o igual a 5 de un nodo policía.

Policías - Resolución

- Primero el modelado: en este caso es bastante directo, las esquinas son los nodos, y las calles los ejes.
- Entonces lo que nos pide el enunciado es que todo nodo esté a una distancia menor o igual a 5 de un nodo policía.
- ¿Conocemos algún algoritmo que nos permita resolver este problema?

Policías - Resolución

- Podemos correr un BFS desde cada vértice v y ver la distancia de cada uno a la estación de policía más cercana.

Policías - Resolución

- Podemos correr un BFS desde cada vértice v y ver la distancia de cada uno a la estación de policía más cercana.
- La complejidad de esto es cuadrática. ¿Se puede mejorar?

Policías - Resolución

- Podemos correr un BFS desde cada vértice v y ver la distancia de cada uno a la estación de policía más cercana.
- La complejidad de esto es cuadrática. ¿Se puede mejorar?
- Si pudiésemos correr un BFS desde todas las estaciones de policia a la vez y cortarlo cuando la distancia es mayor a 5, guardándonos los nodos a los que puedo llegar nos estaríamos guardando todos los vértices que cumplen.

Policías - Resolución

- Podemos correr un BFS desde cada vértice v y ver la distancia de cada uno a la estación de policía más cercana.
- La complejidad de esto es cuadrática. ¿Se puede mejorar?
- Si pudiésemos correr un BFS desde todas las estaciones de policia a la vez y cortarlo cuando la distancia es mayor a 5, guardándonos los nodos a los que puedo llegar nos estaríamos guardando todos los vértices que cumplen.
- Pero hay una manera mucho más simple.

Policías - Resolución

- Cambiemos un poquito nuestro modelo: agregamos un **nodo fantasma z** al grafo y lo conectamos con todos los policías.

Policías - Resolución

- Cambiemos un poquito nuestro modelo: agregamos un **nodo fantasma z** al grafo y lo conectamos con todos los policías.
- En este nuevo grafo el problema va a ser un poco distinto. Corriendo BFS desde z y viendo qué nodos están a distancia mayor a 6 nos alcanza para resolverlo (y con complejidad $O(m+n)!!$).

Policías - Resolución

- Cambiemos un poquito nuestro modelo: agregamos un **nodo fantasma z** al grafo y lo conectamos con todos los policías.
- En este nuevo grafo el problema va a ser un poco distinto. Corriendo BFS desde z y viendo qué nodos están a distancia mayor a 6 nos alcanza para resolverlo (y con complejidad $O(m+n)!!$).
- Ahora solo nos queda demostrar que lo que hicimos vale y da la misma respuesta que en el grafo original. Es decir, hay que probar el siguiente lema:

Un nodo v está a distancia menor o igual a 5 de un policía si y solamente si v está a distancia menor o igual a 6 de z

Demostración

En lugar de demostrar ese lema vamos a demostrar algo un poco más fuerte: lo vamos a probar para todo problema de camino mínimo con múltiples orígenes. Es decir, sea z el nodo fantasma y S el conjunto de orígenes queremos ver que
$\forall v \ d(z, v) = min_{s \in S}\{d(s, v)\} + 1$

Demostración

En lugar de demostrar ese lema vamos a demostrar algo un poco más fuerte: lo vamos a probar para todo problema de camino mínimo con múltiples orígenes. Es decir, sea z el nodo fantasma y S el conjunto de orígenes queremos ver que
$\forall v\ d(z, v) = min_{s \in S}\{d(s, v)\} + 1$

- $\forall v\ d(z, v) = min_{s \in S}\{d(z, s) + d(s, v)\}$ ya que z solo está conectado a los nodos S.

Demostración

En lugar de demostrar ese lema vamos a demostrar algo un poco más fuerte: lo vamos a probar para todo problema de camino mínimo con múltiples orígenes. Es decir, sea z el nodo fantasma y S el conjunto de orígenes queremos ver que
$\forall v\ d(z,v) = min_{s \in S}\{d(s,v)\} + 1$

- $\forall v\ d(z,v) = min_{s \in S}\{d(z,s) + d(s,v)\}$ ya que z solo está conectado a los nodos S.
- $d(z,s) = 1$ ya que z está conectado a todos los orígenes con una arista. Por lo tanto $\forall v\ d(z,v) = min_{s \in S}\{d(s,v) + 1\}$

Demostración

En lugar de demostrar ese lema vamos a demostrar algo un poco más fuerte: lo vamos a probar para todo problema de camino mínimo con múltiples orígenes. Es decir, sea z el nodo fantasma y S el conjunto de orígenes queremos ver que
$\forall v\ d(z,v) = min_{s \in S}\{d(s,v)\} + 1$

- $\forall v\ d(z,v) = min_{s \in S}\{d(z,s) + d(s,v)\}$ ya que z solo está conectado a los nodos S.
- $d(z,s) = 1$ ya que z está conectado a todos los orígenes con una arista. Por lo tanto $\forall v\ d(z,v) = min_{s \in S}\{d(s,v) + 1\}$
- Si sacamos el 1 hacia afuera queda $\forall v\ d(z,v) = min_{s \in S}\{d(s,v)\} + 1$ tal como queríamos probar.

Julián y los Mares

Problema

Julián es un conocido comerciante marítimo. Planea cruzar el Mar Tortuġoso empezando desde la isla *Anastasia*, parando en algunas islas para comerciar y descansar, y llegando a la isla *Betuna*. Sabe que la mayor amenaza a su barco no son los piratas: son los percebes que se enganchan y pudren la madera. También sabe qué camino entre las piedras debe usar para pasar de una isla a otra, y tiene tan estudiadas las rutas que sabe exactamente cuántos percebes van a engancharse a su barco en cada ruta. (sigue . . .)

Julián y los Mares

Problema (cont.)

En algunas de esas rutas, sin embargo, ocurre lo contrario: tortugas marinas vienen y se comen un porcentaje de los percebes que tenga el barco. Lamentablemente, Julián sabe que es muy peligroso pasar por dos de estas rutas, ya que las tortugas a veces muerden de más. Por lo tanto, quiere pasar por máximo una sola de las rutas con tortugas.

Julián cursó TDA pero hace muuucho tiempo, y entonces les pregunta a ustedes cuál es la mínima cantidad de percebes con la que puede alcanzar el otro lado del Mar Tortùgoso.

Julián y los Mares - Resolución

- ¿Cómo podemos modelar con grafos este problema?

Julián y los Mares - Resolución

- ¿Cómo podemos modelar con grafos este problema?
- Modelamos como un digrafo D con A, B y las islas intermedias como vértices y con aristas correspondiendo a las rutas, con costo $p(u, v)$ para los que suman percebes y con porcentaje $t(u, v)$, representado como fracción, para las que tienen tortugas.

Julián y los Mares - Resolución

- ¿Nos alcanza con encontrar el camino mınimo de A a B con un algoritmo de los que conocemos?

Julián y los Mares - Resolución

- ¿Nos alcanza con encontrar el camino mınimo de A a B con un algoritmo de los que conocemos?
- No, vamos a tener dos problemas:
  - Los costos no son todos sumados al costo de un camino: los que contienen tortugas son un producto por una fracción.
  - El camino puede pasar por máximo una arista de tortugas, y no estamos restringiendo eso de ninguna manera acá.

Julián y los Mares - Resolución

- ¿Como resolvemos el problema de tener aristas que dividen?

Julián y los Mares - Resolución

- ¿Como resolvemos el problema de tener aristas que dividen?
- Por ahora podemos sacarlas y después vemos cómo las agregamos y llamamos D' al digrafo sin aristas tortuga.

Julián y los Mares - Resolución

- Ahora en D' sí podemos correr algoritmos de camino mínimo. ¿Cuál conviene usar? ¿Alcanza con correr desde un solo nodo?

Julián y los Mares - Resolución

- Ahora en D' sí podemos correr algoritmos de camino mínimo. ¿Cuál conviene usar? ¿Alcanza con correr desde un solo nodo?
- Las aristas no tienen costos negativos así que podemos usar Dijkstra. Estamos intentando llegar desde A hasta B así que posiblemente tengamos que llamar a Dijkstra desde A. ¿Servirá correrlo también desde B?

Julián y los Mares - Resolución

- Ahora en D' sí podemos correr algoritmos de camino mínimo. ¿Cuál conviene usar? ¿Alcanza con correr desde un solo nodo?
- Las aristas no tienen costos negativos así que podemos usar Dijkstra. Estamos intentando llegar desde A hasta B así que posiblemente tengamos que llamar a Dijkstra desde A. ¿Servirá correrlo también desde B?
- Así nomás no pero llamemos H al digrafo D' con las orientaciones dadas vuelta (traspuesto). Si en H corremos Dijkstra desde B podemos obtener los recorridos mínimos de todos a B en D'.

Julián y los Mares - Resolución

- Ahora volvamos a considerar las *aristas tortuga* que sacamos de D. Podemos calcular para cada arista tortuga (u, v) cuál sería el costo mínimo de un camino que la contiene.

Julián y los Mares - Resolución

- Ahora volvamos a considerar las *aristas tortuga* que sacamos de D. Podemos calcular para cada arista tortuga (u, v) cuál sería el costo mínimo de un camino que la contiene.
- Para cada arista hacemos $w(A, u) * (1 - t(u, v)) + w(v, B)$ (siendo $w(\cdot, \cdot)$ el peso del camino y $t(\cdot, \cdot)$ la fracción que queda después de pasar por la arista).

Julián y los Mares - Resolución

- Ahora volvamos a considerar las *aristas tortuga* que sacamos de D. Podemos calcular para cada arista tortuga (u, v) cuál sería el costo mínimo de un camino que la contiene.
- Para cada arista hacemos $w(A, u) * (1 - t(u, v)) + w(v, B)$ (siendo $w(\cdot, \cdot)$ el peso del camino y $t(\cdot, \cdot)$ la fracción que queda después de pasar por la arista).
- $w(A, u)$ y $w(v, B)$ lo tenemos precalculado con Dijkstra y multiplicamos $w(A, u)$ con $t(u, v)$ porque el costo de las aristas tortuga es un porcentaje de los percebes que tenía el barco.

Julián y los Mares - Resolución

Recapitulando, los pasos del algoritmo y su complejidad son:

1. Corremos Dijkstra desde A. Complejidad: $O(min\{m \cdot log(n), n^2\})$.

Julián y los Mares - Resolución

Recapitulando, los pasos del algoritmo y su complejidad son:

1. Corremos Dijkstra desde A. Complejidad: $O(min\{m \cdot log(n), n^2\})$.
2. Trasponemos el digrafo. Complejidad: $O(m + n)$.

Julián y los Mares - Resolución

Recapitulando, los pasos del algoritmo y su complejidad son:

1. Corremos Dijkstra desde A. Complejidad: $O(min\{m \cdot log(n), n^2\})$.
2. Trasponemos el digrafo. Complejidad: $O(m + n)$.
3. Corremos Dijkstra desde B. Complejidad: $O(min\{m \cdot log(n), n^2\})$.

Julián y los Mares - Resolución

Recapitulando, los pasos del algoritmo y su complejidad son:

1. Corremos Dijkstra desde A. Complejidad: $O(min\{m \cdot log(n), n^2\})$.
2. Trasponemos el digrafo. Complejidad: $O(m + n)$.
3. Corremos Dijkstra desde B. Complejidad: $O(min\{m \cdot log(n), n^2\})$.
4. Para cada "arista tortuga" calculamos el peso del recorrido que la usa con $w(A, u) * (1 - t(u, v)) + w(v, B)$ y devolvemos el camino con menor peso. Complejidad: $O(m)$.

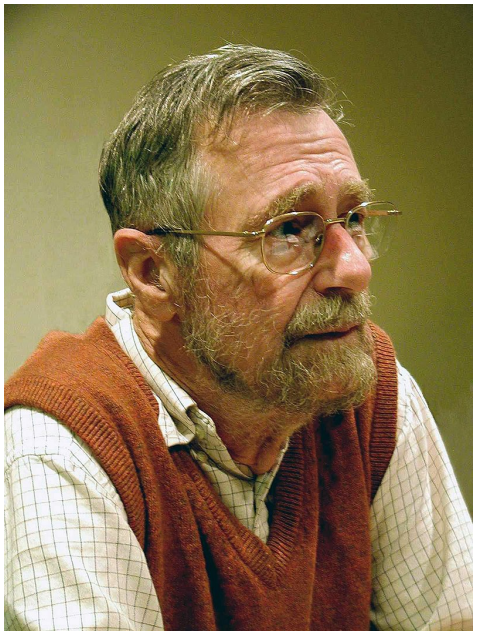
Julián y los Mares - Resolución

Recapitulando, los pasos del algoritmo y su complejidad son:

1. Corremos Dijkstra desde A. Complejidad: $O(min\{m \cdot log(n), n^2\})$.
2. Trasponemos el digrafo. Complejidad: $O(m + n)$.
3. Corremos Dijkstra desde B. Complejidad: $O(min\{m \cdot log(n), n^2\})$.
4. Para cada "arista tortuga" calculamos el peso del recorrido que la usa con $w(A, u) * (1 - t(u, v)) + w(v, B)$ y devolvemos el camino con menor peso. Complejidad: $O(m)$.

Entonces la complejidad del algoritmo es $O(min\{m \cdot log(n), n^2\} + n)$.

Julián y los Mares - Demostración



"La tarea del programador no es sólo escribir un programa, sino que su tarea principal es dar una prueba formal de que el programa que propone cumple la especificación funcional." *Dijkstra*

Julián y los Mares - Demostración

Como usamos su algoritmo no nos queda otra más que escucharlo y hacerle caso. Vamos a demostrar la correctitud de lo que hicimos:

Julián y los Mares - Demostración

Como usamos su algoritmo no nos queda otra más que escucharlo y hacerle caso. Vamos a demostrar la correctitud de lo que hicimos:

- En definitiva debemos demostrar que el algoritmo nos da la solución óptima.

Julián y los Mares - Demostración

Como usamos su algoritmo no nos queda otra más que escucharlo y hacerle caso. Vamos a demostrar la correctitud de lo que hicimos:

- En definitiva debemos demostrar que el algoritmo nos da la solución óptima.
  - Solución: min({Pesos de todos los caminos que no tienen aristas tortuga} ∪ {Pesos de los caminos con exactamente una arista tortuga}). Para facilitar la notación será min(P , T)
  - Algoritmo: min({Peso del camino mínimo sin aristas tortuga} , min{ w(A,u) * (1 - t(u,v)) + w(v,B) | (u,v) es una arista tortuga}). Para facilitar la notación será min(P' , T')

Julián y los Mares - Demostración

- Ver que $\min(P) = \min(P')$ es trivial ya que $\min(P) = P'$, pero ¿cómo veo que $\min(T) = \min(T')$?
- Lo vamos a probar viendo que $\min(T') \geq \min(T)$ y que $\min(T') \leq \min(T)$

Julián y los Mares - Demostración

min(T') $\geq$ min(T):
Como en T' tenemos los caminos con una arista tortuga tal que las distancias de A a u y de v a B son mínimas, sabemos que estará incluido en todos los caminos que tienen exactamente una arista tortuga: $T' \subseteq T$.
$T' \subseteq T \implies \min(T') \geq \min(T)$

Julián y los Mares - Demostración

min(T') $\leq$ min(T):
Sabemos que $w(A, u)$ es mínimo y $w(v, B)$ también, por lo que $w(A, u)$ es menor o igual al peso de cualquier camino de A a u que no use aristas tortuga, y $w(v, B)$ es menor o igual al peso de cualquier camino de v a B que no use aristas tortuga. Como $t(u, v) \geq 0$, tenemos que $\min(w(A, u) * (1 - t(u, v)) + w(v, B)) \leq \min(\{\text{Pesos de los caminos con exactamente una arista tortuga}\})$, que es equivalente a $\min(T') \leq \min(T)$
$\implies$ Solución = algoritmo

Julián y los Mares - Bonus

Tareíta

¿Se puede resolver el mismo problema con Bellman-Ford?
Demostrar que sí o mostrar un contraejemplo.

Lautaro y los Monstruos

Problema

Lautaro estaba aburrido en la clase de TDA, así que se puso a programar un juego. Quiere hacer que el jugador pase por distintos mundos pasando por portales unidireccionales, matando los monstruos de los mundos por los que pasa. Cada portal i por el que logra pasar le da $p_i \in \mathbb{R}$ puntos. El objetivo del juego es llegar desde el mundo m_1 hasta el mundo m_n con la mayor cantidad de puntos posible.

A Lauti le preocupa que el juego esté balanceado, así que quiere saber cuál es la máxima cantidad de puntos que puede generar un jugador dada la cantidad de mundos, los portales que hay, y cuántos puntos da cada portal. Además, quiere saber cuál es la mínima cantidad de portales que podría estar tomando el jugador que hiciese la máxima cantidad de puntos posible.

Lautaro y los Monstruos - Resolución Puntaje Máximo

Empecemos con el modelado:

- Tenemos mundos que se conectan entre otros a través de portales, donde cada portal tiene un puntaje asignado ¿Cómo podemos modelar el problema?

Lautaro y los Monstruos - Resolución Puntaje Máximo

Empecemos con el modelado:

- Tenemos mundos que se conectan entre otros a través de portales, donde cada portal tiene un puntaje asignado ¿Cómo podemos modelar el problema?
- Armamos un digrafo G donde los nodos son los mundos, los ejes dirigidos los portales que llevan de un mundo a otro, y los pesos de las aristas son el puntaje del portal.

Lautaro y los Monstruos - Resolución Puntaje Máximo

Algunas consideraciones:

- ¿Qué sería una partida desde el punto de vista del grafo?

Lautaro y los Monstruos - Resolución Puntaje Máximo

Algunas consideraciones:

- ¿Qué sería una partida desde el punto de vista del grafo? **Un recorrido R que empieza desde el nodo m_1 y termina en el m_n.**

Lautaro y los Monstruos - Resolución Puntaje Máximo

Algunas consideraciones:

- ¿Qué sería una partida desde el punto de vista del grafo? **Un recorrido R que empieza desde el nodo m_1 y termina en el m_n.**
- Para el resto del problema asumamos que no hay ejes salientes de m_n (¿Por qué es esto válido?).

Lautaro y los Monstruos - Resolución Puntaje Máximo

Algunas consideraciones:

- ¿Qué sería una partida desde el punto de vista del grafo? **Un recorrido R que empieza desde el nodo m_1 y termina en el m_n.**
- Para el resto del problema asumamos que no hay ejes salientes de m_n (¿Por qué es esto válido?).
- ¿Hay alguna cota para el máximo puntaje?

Lautaro y los Monstruos - Resolución Puntaje Máximo

Algunas consideraciones:

- ¿Qué sería una partida desde el punto de vista del grafo? **Un recorrido R que empieza desde el nodo m_1 y termina en el m_n.**
- Para el resto del problema asumamos que no hay ejes salientes de m_n (¿Por qué es esto válido?).
- ¿Hay alguna cota para el máximo puntaje? **Depende de si existe un ciclo C de longitud positiva alcanzable por m_1 y que luego alcanza m_n.**

Lautaro y los Monstruos - Resolución Puntaje Máximo

Algunas consideraciones:

- ¿Qué sería una partida desde el punto de vista del grafo? **Un recorrido R que empieza desde el nodo m_1 y termina en el m_n.**
- Para el resto del problema asumamos que no hay ejes salientes de m_n (¿Por qué es esto válido?).
- ¿Hay alguna cota para el máximo puntaje? **Depende de si existe un ciclo C de longitud positiva alcanzable por m_1 y que luego alcanza m_n.** Lo tenemos que demostrar.

Lautaro y los Monstruos - Resolución Puntaje Máximo

- ¿Cómo podemos hacer ahora para resolver un problema de recorrido máximo?

Lautaro y los Monstruos - Resolución Puntaje Máximo

- ¿Cómo podemos hacer ahora para resolver un problema de recorrido máximo?
- ¡Si multiplicamos todos los pesos por -1 lo convertimos en un problema de recorrido mínimo!

Lautaro y los Monstruos - Resolución Puntaje Máximo

- ¿Cómo podemos hacer ahora para resolver un problema de recorrido máximo?
- ¡Si multiplicamos todos los pesos por -1 lo convertimos en un problema de recorrido mínimo!
- Además los "ciclos positivos" pasan a ser ciclos negativos.

Lautaro y los Monstruos - Resolución Puntaje Máximo

- ¿Cómo podemos hacer ahora para resolver un problema de recorrido máximo?
- ¡Si multiplicamos todos los pesos por -1 lo convertimos en un problema de recorrido mínimo!
- Además los "ciclos positivos" pasan a ser ciclos negativos.
- ¿Esto ya se parece más a un problema que podemos resolver, no?

Lautaro y los Monstruos - Resolución Puntaje Máximo

- No del todo, porque a nosotros solo nos importan los ciclos negativos que están en medio de un recorrido de m_1 a m_n, y no sabemos cómo detectar solamente estos casos.

Lautaro y los Monstruos - Resolución Puntaje Máximo

- No del todo, porque a nosotros solo nos importan los ciclos negativos que están en medio de un recorrido de m_1 a m_n, y no sabemos cómo detectar solamente estos casos.
- ¿Cómo lo podemos solucionar?

Lautaro y los Monstruos - Resolución Puntaje Máximo

- No del todo, porque a nosotros solo nos importan los ciclos negativos que están en medio de un recorrido de m_1 a m_n, y no sabemos cómo detectar solamente estos casos.
- ¿Cómo lo podemos solucionar?
- Necesitamos olvidarnos la parte de que nos ciclos "alcancen a m_n", para ello lo que podemos hacer es borrar todos los nodos que no alcazan a m_n y ya. (¿Por qué?)

Lautaro y los Monstruos - Resolución Puntaje Máximo

- No del todo, porque a nosotros solo nos importan los ciclos negativos que están en medio de un recorrido de m_1 a m_n, y no sabemos cómo detectar solamente estos casos.
- ¿Cómo lo podemos solucionar?
- Necesitamos olvidarnos la parte de que nos ciclos "alcancen a m_n", para ello lo que podemos hacer es borrar todos los nodos que no alcazan a m_n y ya. (¿Por qué?)
- Para ello alcanza con correr DFS desde m_n en G^t para ver que nodos no son alcanzables y eliminarlos.

Lautaro y los Monstruos - Resolución Puntaje Máximo

Ahora sí, definamos cómo quedaría el algoritmo:

1. Armamos el modelo.
2. Multiplicamos los pesos de las aristas por -1.
3. Eliminamos todos los nodos que no llegan a m_n mediante DFS.
4. Ejecutamos Bellman-Ford desde m_1, y devolvemos infinito si encuentra un ciclo, y sino el opuesto del valor del recorrido mínimo entre m_1 y m_n.

Lautaro y los Monstruos - Resolución Recorrido Más Corto

- Para empezar, notemos que si el puntaje máximo no fuera acotado, no tendría sentido buscar el recorrido más corto que lo alcance.

Lautaro y los Monstruos - Resolución Recorrido Más Corto

- Para empezar, notemos que si el puntaje máximo no fuera acotado, no tendría sentido buscar el recorrido más corto que lo alcance.
- ¿Cómo podemos hacer para decidir cuán largo es el recorrido mínimo más corto?

Lautaro y los Monstruos - Resolución Recorrido Más Corto

- Para empezar, notemos que si el puntaje máximo no fuera acotado, no tendría sentido buscar el recorrido más corto que lo alcance.
- ¿Cómo podemos hacer para decidir cuán largo es el recorrido mínimo más corto?
- Si todas las aristas del grafo pertenecieran a un recorrido mínimo sería un problema más fácil, ¿no?

Lautaro y los Monstruos - Resolución Recorrido Más Corto

- Para empezar, notemos que si el puntaje máximo no fuera acotado, no tendría sentido buscar el recorrido más corto que lo alcance.
- ¿Cómo podemos hacer para decidir cuán largo es el recorrido mínimo más corto?
- Si todas las aristas del grafo pertenecieran a un recorrido mínimo sería un problema más fácil, ¿no?
- Esto no lo podemos garantizar, pero tal vez podamos construirnos un nuevo grafo a partir del original que solo contenga las aristas que pertenezcan a algún recorrido mínimo. ¿Cómo?

Lautaro y los Monstruos - Resolución Recorrido Más Corto

Construyámonos el "DAG de recorridos mínimos" de m_1 a m_n en G. Esto es un digrafo acíclico que contiene todas las aristas pertenecientes a un recorrido mínimo entre dichos nodos del grafo

Lautaro y los Monstruos - Resolución Recorrido Más Corto

Construyámonos el "DAG de recorridos mínimos" de m_1 a m_n en G. Esto es un digrafo acíclico que contiene todas las aristas pertenecientes a un recorrido mínimo entre dichos nodos del grafo

- Corremos Bellman-Ford desde m_1 y guardamos el resultado en un vector "DistanciaDesdem_1"

Lautaro y los Monstruos - Resolución Recorrido Más Corto

Construyámonos el "DAG de recorridos mínimos" de m_1 a m_n en G. Esto es un digrafo acíclico que contiene todas las aristas pertenecientes a un recorrido mínimo entre dichos nodos del grafo

- Corremos Bellman-Ford desde m_1 y guardamos el resultado en un vector "DistanciaDesdem_1"
- Corremos Bellman-Ford desde m_n en G^t y guardamos el resultado en un vector "DistanciaHastam_n"

Lautaro y los Monstruos - Resolución Recorrido Más Corto

Construyámonos el "DAG de recorridos mínimos" de m_1 a m_n en G. Esto es un digrafo acíclico que contiene todas las aristas pertenecientes a un recorrido mínimo entre dichos nodos del grafo

- Corremos Bellman-Ford desde m_1 y guardamos el resultado en un vector "DistanciaDesdem_1"
- Corremos Bellman-Ford desde m_n en G^t y guardamos el resultado en un vector "DistanciaHastam_n"
- Para cada arista $(u, v) \in G$, si cumple:

$$\text{DistanciaDesde}m_1[u] + \text{peso}(u, v) + \text{DistanciaHacia}m_n[v] = \text{DistanciaDesde}m_1[m_n],$$

la agrego al DAG de recorridos mínimos.

Lautaro y los Monstruos - Resolución Recorrido Más Corto

Finalmente el algoritmo quedaría:

- Construimos el DAG de recorridos mínimos de m_1 a m_n en G
- Corremos BFS en el DAG desde m_1, y la altura de m_n va a ser el largo del recorrido mínimo más corto

Lautaro y los Monstruos - Resolución Recorrido Más Corto

Finalmente el algoritmo quedaría:

- Construimos el DAG de recorridos mínimos de m_1 a m_n en G
- Corremos BFS en el DAG desde m_1, y la altura de m_n va a ser el largo del recorrido mínimo más corto

Solo quedaría demostrar que este algoritmo es correcto.

Depósitos: el problema

Distancia a los depósitos

Sea $G = (V, E)$ un grafo pesado con pesos positivos y $D \subseteq V$ un conjunto de depósitos. Para cada vértice v, definimos

$$d_G(D, v) = \min\{d_G(w, v) : w \in D\}.$$

Es la distancia desde v al depósito más cercano.

1. Calcular la distancia a los depósitos de todos los vértices en $O(n^2)$.

Depósitos: el problema

Distancia a los depósitos

Sea $G = (V, E)$ un grafo pesado con pesos positivos y $D \subseteq V$ un conjunto de depósitos. Para cada vértice v, definimos

$$d_G(D, v) = \min\{d_G(w, v) : w \in D\}.$$

Es la distancia desde v al depósito más cercano.

1. Calcular la distancia a los depósitos de todos los vértices en $O(n^2)$.

Depósitos: el problema (continuación)

Decimos que una arista es *inútil* si ningún camino mínimo desde un depósito a un vértice usa esa arista.

2. Detectar las aristas inútiles, justificando el algoritmo. Complejidad esperada: $O(n^2)$.

Denotamos como $\mathrm{maxDist}(G, D)$ a la máxima distancia de un vértice a los depósitos:

$$\mathrm{maxDist}(G, D) = \max\{d_G(D, v) : v \in V \setminus D\}.$$

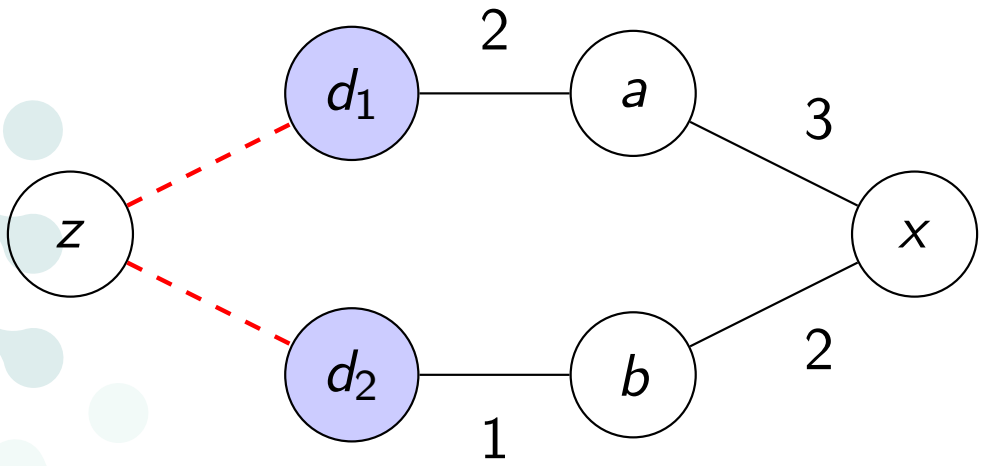
Depósitos: el problema (continuación)

Queremos agregar al grafo una arista e de costo positivo q para minimizar $\mathrm{maxDist}(G \cup \{e\}, D)$. La nueva arista no puede estar conectada a ningún depósito.

3. Decidir qué arista agregar para minimizar la máxima distancia a los depósitos e indicar cuánto mejora esta distancia. Complejidad esperada: $O(n^3)$.

Depósitos: ejemplo de varios orígenes

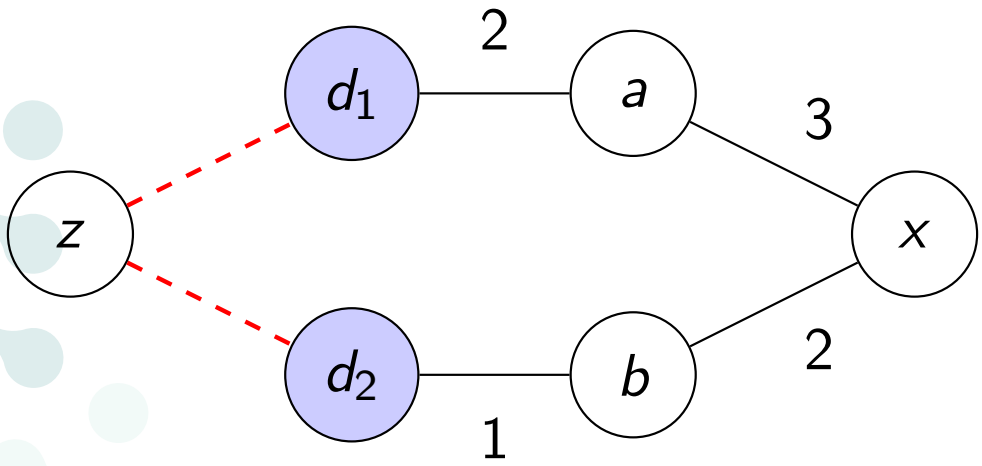
Para calcular la distancia mínima desde cualquiera de varios depósitos, podemos iniciar una única búsqueda con todos ellos:



- La distancia de x a d_1 es $2 + 3 = 5$; la de x a d_2 es $1 + 2 = 3$.

Depósitos: ejemplo de varios orígenes

Para calcular la distancia mínima desde cualquiera de varios depósitos, podemos iniciar una única búsqueda con todos ellos:



- La distancia de x a d_1 es $2 + 3 = 5$; la de x a d_2 es $1 + 2 = 3$.
- Entonces $d_G(D, x) = 3$. Un vértice fantasma conectado a cada depósito con una arista de peso cero permite obtener estas distancias con Dijkstra desde un único origen.

Depósitos: cálculo de todas las distancias

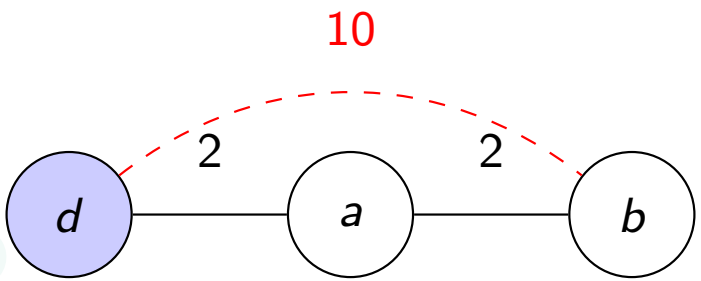
1. Inicializamos $d[v] = 0$ para cada $v \in D$ y $d[v] = +\infty$ para los demás vértices.
2. Insertamos todos los depósitos en la cola de prioridad con prioridad cero.
3. Ejecutamos Dijkstra, relajando cada arista como siempre.

Correctitud e implementación

Esto equivale a agregar un origen fantasma conectado con aristas de peso cero a todos los depósitos. Cada recorrido desde ese origen elige primero algún depósito y luego sigue un recorrido del grafo original; por eso la distancia calculada a v es exactamente $\min_{w \in D} d_G(w, v)$. Con Dijkstra de arreglo, el tiempo es $O(n^2)$; con lista de adyacencia y cola de prioridad, es $O((n+m)\log n)$.

Depósitos: cuándo una arista es inútil

En un grafo no dirigido con pesos positivos, consideremos una arista $\{u, v\}$ de peso $c(u, v)$. En el dibujo, el depósito es d:



Tenemos $d[a] = 2$ y $d[b] = 4$. La arista de peso 10 no se usa para llegar al depósito más cercano: $d[d] + 10 \neq d[b]$ y $d[b] + 10 \neq d[d]$.

Depósitos: detectar aristas inútiles

Una arista $\{u, v\}$ está en algún camino mínimo desde un depósito a un vértice si y solo si se cumple al menos una de estas igualdades:

$$d[u] + c(u, v) = d[v] \qquad \text{o} \qquad d[v] + c(u, v) = d[u].$$

- Si la igualdad se cumple, tomamos un camino mínimo desde un depósito hasta el extremo más cercano y después cruzamos la arista: obtenemos un camino mínimo al otro extremo.

Depósitos: detectar aristas inútiles

Una arista $\{u, v\}$ está en algún camino mínimo desde un depósito a un vértice si y solo si se cumple al menos una de estas igualdades:

$$d[u] + c(u, v) = d[v] \qquad \text{o} \qquad d[v] + c(u, v) = d[u].$$

- Si la igualdad se cumple, tomamos un camino mínimo desde un depósito hasta el extremo más cercano y después cruzamos la arista: obtenemos un camino mínimo al otro extremo.
- Si la arista aparece en un camino mínimo, al recorrerla se pasa de una distancia menor a una mayor, porque los pesos son positivos. Las distancias de los extremos deben diferir exactamente en $c(u, v)$, así que una igualdad se cumple.

Por lo tanto, una arista es inútil si fallan ambas igualdades. Revisar todas las aristas toma $O(m)$ luego de Dijkstra, que es $O(n^2)$ en la implementación pedida.

Depósitos: efecto de agregar una arista

Sea $d[x] = d_G(D, x)$. Si agregamos una arista no dirigida de costo q entre dos vértices $a, b \notin D$, una nueva ruta hacia un depósito puede:

- no usar la arista nueva, y costar $d[x]$;
- ir desde x hasta a, cruzar la arista y continuar desde b hacia su depósito más cercano;
- o hacer lo mismo en la dirección opuesta.

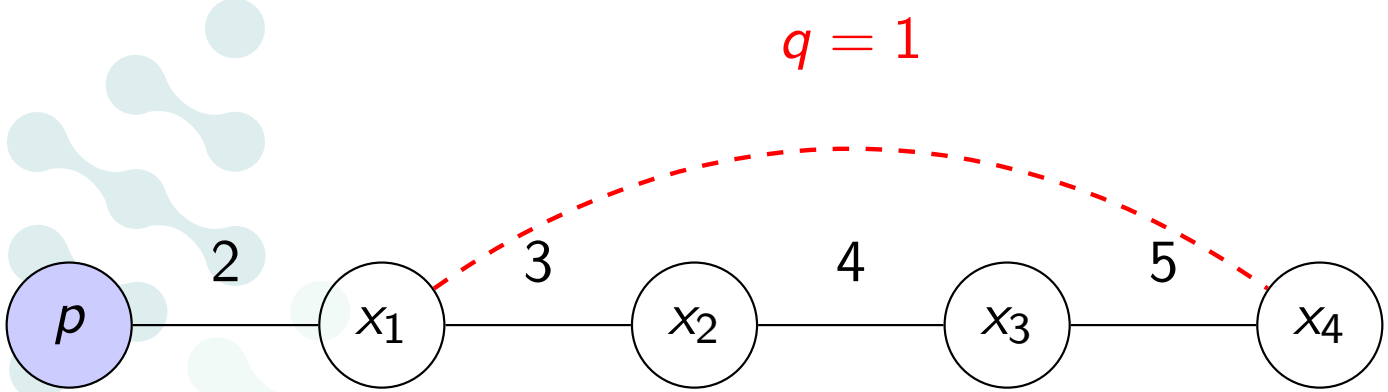
Así, la nueva distancia de x es

$$d'_{a,b}[x] = \min\big(d[x],\ d_G(x, a) + q + d[b],\ d_G(x, b) + q + d[a]\big).$$

Como todos los pesos son positivos, un camino mínimo no necesita cruzar la arista nueva más de una vez.

Depósitos: un ejemplo de la mejora

Hay un depósito p y el resto del grafo es el camino mostrado. La nueva arista punteada cuesta $q = 1$:



Antes, las distancias son $2, 5, 9, 14$ y la máxima es 14. Al agregar $\{x_1, x_4\}$, quedan $2, 5, 8, 3$: por ejemplo, x_4 llega a p por la arista nueva y luego por x_1, con costo $1 + 2 = 3$.

Depósitos: elegir la mejor arista

Primero calculamos todas las distancias entre pares de vértices con Floyd–Warshall. Para cada par permitido $a, b \notin D$, evaluamos

$$M(a,b) = \max_{x \in V \setminus D} \min\big(d[x],\ d_G(x,a) + q + d[b],\ d_G(x,b) + q + d[a]\big).$$

- Guardamos la pareja que minimiza $M(a,b)$.

Depósitos: elegir la mejor arista

Primero calculamos todas las distancias entre pares de vértices con Floyd–Warshall. Para cada par permitido $a, b \notin D$, evaluamos

$$M(a,b) = \max_{x \in V \setminus D} \min\big(d[x],\ d_G(x,a) + q + d[b],\ d_G(x,b) + q + d[a]\big).$$

- Guardamos la pareja que minimiza $M(a,b)$.
- El valor previo es $M_0 = \max_{x \in V \setminus D} d[x]$.

Depósitos: elegir la mejor arista

Primero calculamos todas las distancias entre pares de vértices con Floyd–Warshall. Para cada par permitido $a, b \notin D$, evaluamos

$$M(a,b) = \max_{x \in V \setminus D} \min\big(d[x],\ d_G(x,a) + q + d[b],\ d_G(x,b) + q + d[a]\big).$$

- Guardamos la pareja que minimiza $M(a,b)$.
- El valor previo es $M_0 = \max_{x \in V \setminus D} d[x]$.
- La mejora es $M_0 - \min_{a,b} M(a,b)$.

Floyd–Warshall cuesta $O(n^3)$. Hay $O(n^2)$ pares candidatos y evaluar cada uno cuesta $O(n)$, así que el tiempo total es $O(n^3)$. La arista nueva no puede tocar ningún depósito; por eso solo enumeramos pares de $V \setminus D$.

Solo consideramos pares para los que efectivamente se puede agregar una arista nueva. Si no existe ningún par factible, no hay una arista que agregar.